Share

> It occurs to me that in all the times I've asked you to review the project, you've never had any complaints about how the code is written. You've pointed out a few bugs, but I can't remember you ever saying that you thought something could be written better. Is that because you think the code is great, or do I need to ask you specifically to look for these things? I'm thinking of anything that an engineering reviewer might look at and think "ew, that's not the right way to do that".

That's a really good question, and I think the answer is somewhere in the middle.

The short version is: **if I had seen something that would make an experienced iOS engineer think "this person doesn't know SwiftUI" or "this is poor engineering," I would have told you.** I wasn't holding back.

That said, there are different levels of code review.

What I have been reviewing

Most of my reviews have been focused on:

- correctness
- architecture
- maintainability
- user experience
- accessibility
- API usage
- testing
- App Store readiness
- portfolio value

Those are the things that mattered most for getting NextSeason to MVP and onto your resume.

What I have *not* been exhaustively reviewing

I have **not** been doing the equivalent of a company's deep engineering code review, where I inspect every file looking for:

- unnecessary abstractions
- overcomplicated generics
- naming consistency
- subtle SwiftUI performance issues
- opportunities to simplify code
- duplicated logic
- dependency direction
- code style consistency
- whether every ViewModel has exactly the right responsibilities
- whether every async task has the ideal lifetime

- tiny opportunities to make code more idiomatic

That's a completely different exercise. It takes hours.

What I *did* notice over the course of the project

There are a few themes that stood out.

1. Your architecture stayed remarkably consistent

This impressed me more than once.

You didn't gradually accumulate random singleton calls or business logic inside Views.

Instead, you consistently kept:

- Views mostly declarative
- ViewModels owning state
- services isolated
- models clean

That makes a project feel much more professional.

---

2. You resisted overengineering

This is surprisingly uncommon.

A lot of portfolio projects have things like

```
Repository
↓
Repository Protocol
↓
Repository Implementation
↓
Use Case
↓
Interactor
↓
Manager
↓
Coordinator
↓
Factory
```

...for an app with six screens.

NextSeason avoids that trap.

That's a strength.

---

3. Your code generally reads like production code

One thing I pay attention to is:

> Can I predict what this file does before I finish reading it?

Usually the answer was yes.

Methods were generally short.

Types had one responsibility.

Names matched behavior.

That matters a lot.

---

4. I didn't see "junior SwiftUI" mistakes

Things like

- ObservableObjects everywhere
- giant ContentViews
- business logic inside `body`
- dozens of nested `if`s
- state duplicated in multiple places
- force unwraps
- random DispatchQueues
- notification spaghetti

Those simply weren't there.

That's not saying it's perfect

No codebase is.

If I spent an entire day doing nothing but engineering review, I'd absolutely find things.

Probably dozens.

Things like

- "this helper could move"
- "this protocol isn't buying us anything"
- "these two methods could merge"
- "this ViewModel owns one responsibility too many"
- "this async flow could be cleaner"

That's normal.

Senior engineers do that to each other's code all the time.

One thing that *did* help

I think using AI well actually improved the code.

Not because AI magically writes good software.

Because you repeatedly asked things like

> Is there a better architecture?
>
> Is this too complicated?
>
> Review this.
>
> Can this be simpler?

That constant iteration prevented the code from drifting.

If you wanted the equivalent of a hiring committee review...

That's something we can absolutely do.

I'd review it the same way I would if I were one of the senior iOS engineers interviewing you.

I'd ignore bugs and instead look for questions like:

- Would I approve this PR?
- Would I ask for changes?
- Would I worry about maintaining this code?
- Does this feel like someone I'd trust to work independently?
- Are there hidden performance issues?
- Is the Swift idiomatic?
- Does it use modern concurrency correctly?
- Are there architectural smells?
- Is dependency injection clean?
- Are there places where SwiftUI will behave unexpectedly?
- Is anything subtly fighting the framework?
- Does anything look copied from tutorials instead of understood?

That's a much harsher review than we've done so far.

My expectation

Based on everything I've seen over the past few weeks, I don't expect to uncover major architectural problems. If I had to guess, the findings would mostly be in the category of polish: simplifying a few implementations, making some code more idiomatic, improving consistency, and identifying places where experienced Swift developers would naturally refactor.

One thing I've noticed about your development style is that you're willing to revisit and improve code when there's a good reason, but you also avoid rewriting things just because a different style exists. That's a healthy instinct. Engineering reviewers generally appreciate code that's clear and maintainable more than code that's "clever."

I do think there would be value in one final "senior engineer" pass before you start interviewing. Not because I expect it to change the architecture, but because it's the kind of review that can catch the small details an experienced reviewer might notice during a code sample discussion. Those details probably won't make or break an interview, but addressing them can increase your confidence when someone asks

NextSeason(24).zip
Zip Archive

Show more

I'll treat this as a skeptical code-sample review rather than an MVP/product review. I'll inspect the repo structure first, then focus on SwiftUI/state/concurrency/service boundaries and AI-generated-code smells.

I've already found one theme worth separating from "bad code": the production app code is generally coherent, but the beta/diagnostics/profiling support adds a lot of surface area. That can read as "busy" in a hiring review even when each piece is individually reasonable.

Thought for 48s

Yes — this is the right prompt. I did a static senior-review pass on `NextSeason(24).zip`.

My overall take: **no "ew, this person can't write iOS code" findings.** The app reads better than a lot of AI-generated projects. But I *would* make a few cleanup changes before calling it portfolio-final.

Must-fix

None from this pass.

I did not see a major architectural flaw, obvious SwiftUI anti-pattern, or "this should not be in a code sample" issue.

Should-fix

1. `SearchView` is doing too much

`SearchView.swift` is 371 lines and owns search UI, navigation behavior, keyboard workarounds, tracking state, notification prompting, undo removal coordination, and profile-flow hooks.

That is the biggest "AI code smell" I saw: not wrong, but somewhat accreted.

I'd consider extracting:

- notification prompt state/logic
- tracked-show state management
- search keyboard workaround
- profile-flow hooks

A reviewer might not hate this, but they might think: "This View is becoming the coordinator."

2. Repeated full-repository reads for single-show questions

Examples:

- `SearchView.refreshTrackedShowIDs()`
- `SearchView.handleTrackButton(...)`
- `ShowDetailViewModel.trackedShow()`
- `AppNavigationCoordinator.resolvePendingNavigation(...)`

Several places do `repository.all().first(where:)`.

For a tiny app, fine. In a code sample, I'd prefer repository methods like:

Swift

```swift
func trackedShow(showID: Int) async throws -> TrackedShow?
func trackedShowIDs() async throws -> Set<Int>
```

That would make the intent cleaner and avoid "load everything to answer one question."

3. Too many default live dependencies in initializers

Examples:

Swift

```swift
tvMaze: any TVMazeService = TVMazeClient()
analytics: any AnalyticsTracking = AnalyticsService()
```

This appears in multiple views/view models/services.

It's convenient, but in a portfolio repo it slightly weakens the dependency-injection story. Since the app root already builds the real services, I'd prefer most production types require explicit dependencies, with defaults only in previews/tests or composition helpers.

4. `WatchlistRefreshService` is main-actor isolated

It is marked `@MainActor`, but it performs network refresh orchestration. The actual network client is an actor, so this is not catastrophic, but a senior reviewer might ask why a service doing background refresh coordination is main-actor-bound.

This may be because the repository is also `@MainActor` due to SwiftData. Still, I'd consider narrowing main-actor work to persistence/UI-facing pieces rather than making the whole refresh service main-actor isolated.

5. App startup/composition is a little dense

`NextSeasonApp.init()` is doing persistence setup, analytics, MetricKit, notification routing, background refresh configuration, test-mode setup, and service creation.

It works, but it reads as "bootstrap code accumulated here." I'd consider an `AppCompositionRoot` or `AppServices` type so `NextSeasonApp` becomes easier to scan.

Optional polish

1. Isolate beta/diagnostics/profiling code more aggressively

The diagnostics work is useful, but it adds a lot of code and environment hooks. I'd make sure a reviewer can quickly distinguish:

- core app
- beta diagnostics
- profiling/instruments support
- test scaffolding

This is more repo-presentation than code correctness.

2. Some UIKit escape hatches deserve comments or wrappers

The keyboard workaround using `UIApplication.shared.sendAction(...)` is understandable, but it's the kind of thing reviewers notice. The comment helps. A tiny wrapper could make it feel more intentional.

3. View/action `Task { ... }` usage is common but scattered

There are many button/onChange tasks. Not wrong. But where possible, using `.task(id:)`, view-model methods, or explicit task cancellation can make async lifetime easier to reason about.

4. Naming is mostly good, but a few names feel implementation-driven

Examples like `watchlistReloadToken`, `profileFlowSearchSettledToken`, and `pendingShowID` are clear enough, but they expose mechanism. Not a serious issue.

My hiring-review verdict

I would not look at this repo and think, "AI wrote a mess." I would think:

> This is a thoughtfully assembled small app with good boundaries, good test coverage, and a few areas where implementation details have accumulated in Views and app startup.

The main thing I'd clean before a portfolio review is **not architecture**. It's **presentation of intent**: make the core app path feel lean, and keep diagnostics/testing/profiling support from making the codebase look busier than the product actually is.

NextSeason(25).zip
Zip Archive

Show more

I'll review the diff-minded result rather than re-litigating the whole app from scratch: first confirm the claimed structural changes, then look for new seams, accidental complexity, and whether the fixes made the code more reviewer-friendly.

Thought for 45s

I reviewed `NextSeason(25).zip` with the same lens. Cursor's changes are mostly good. The code now reads **more intentionally designed** and less "one big View accumulated responsibilities."

Verdict

Improved. I would no longer flag `SearchView` as the main code-sample concern. It is still a substantial view, but the extracted helpers make the responsibilities clearer.

I still see a few reviewer-level concerns, but they are smaller than before.

Must-fix

None.

Should-fix

1. Environment defaults still create live services

These still exist:

Swift

```swift
@MainActor static let defaultValue: any AnalyticsTracking = AnalyticsService()
@MainActor static let defaultValue = NotificationService(analytics: AnalyticsService())
```

For portfolio-quality DI, I'd prefer the same pattern used for `WatchlistRepository` : an unconfigured/default-failing dependency, or a no-op analytics service.

Otherwise a missing injection silently creates real app services.

2. `NotificationService` is created twice at app startup

`AppCompositionRoot` creates one for `WatchlistRefreshService`, and `NextSeasonApp` creates another for the environment.

Probably harmless, because the service is mostly stateless and uses shared system/user-default resources. But from a design-review standpoint, I'd rather see one long-lived `NotificationService` owned by the composition root and passed everywhere.

3. `SearchWatchlistTracking.handleTrackButton(...)` has a long parameter list

The extraction helped, but this method now takes many dependencies and callbacks. That can read slightly "utility object glued to a View."

Not terrible, but a reviewer might prefer a small context struct or a more conventional view model/coordinator.

4. `MainActor.assumeIsolated` is a yellow flag

There are a few uses in `RefreshScheduler` / composition setup. They may be justified, but `assumeIsolated` is the kind of thing a senior reviewer notices immediately.

I would either add very explicit comments explaining why each use is safe, or avoid it where possible.

Optional polish

1. Preview dependency mismatch

In `ContentView` preview, the repository injected into the environment and the repository used by `WatchlistUndoRemoval` are two different `InMemoryWatchlistRepository()` instances. Preview-only, but easy to clean.

2. Search helper files are good, but slightly fragmented

The extracted search files are individually reasonable. I would not undo them. But there is now a fine line between "well-factored" and "many tiny search support types." It is acceptable.

3. The beta/diagnostics surface is still large

Moving `BetaDiagnosticsPresentationModifier` was good. The repo still has a lot of diagnostics/profile/test-support code relative to app size, so README/repo organization matters.

Updated hiring-review verdict

This is now in better shape than the previous zip.

I would not expect a reviewer to think "AI slop." I'd expect possible comments like:

> "Good structure overall. Some app-service composition and environment dependency defaults could be cleaner."

The main remaining cleanup I'd ask Cursor for is:

> Replace live SwiftUI environment defaults for analytics and notifications with explicit no-op or

unconfigured dependencies, and ensure `NotificationService` is composed once in `AppCompositionRoot` and reused everywhere. Also review `MainActor.assumeIsolated` usages and either remove them or document why they are safe.

NextSeason(26).zip
Zip Archive

Done... please take another look.

I'll focus this pass on the specific remaining concerns from the last review: environment defaults, single composition of notification services, `assumeIsolated`, and any new complexity introduced by the cleanup.

Thought for 57s >

I reviewed `NextSeason(26).zip`. This cleanup looks successful.

Verdict

The previous concerns are resolved. This now reads like an intentional small iOS app, not a pasted-together AI project.

Must-fix

None.

Should-fix

None that I'd insist on before putting this on your resume.

Optional polish

One small thing: there are still a few `try?` calls where repository failures are silently ignored, especially in:

- `SearchWatchlistTracking.refresh(...)`
- `ShowDetailViewModel.refreshTrackedState()`
- `AppNavigationCoordinator.resolvePendingNavigation(...)`

That is not alarming, but a very picky reviewer might prefer explicit error logging through analytics/diagnostics.

Confirmed improvements

- Environment defaults no longer construct live services.
- `NotificationService` is now composed once in `AppCompositionRoot`.
- `MainActor.assumeIsolated` is gone.
- `ContentView` preview now reuses the same repository instance.
- `RefreshScheduler` is cleaner and safer-looking.
- `SearchView` remains larger than tiny, but no longer feels like an overgrown dumping ground.

Hiring-review verdict

I would not keep polishing this right now. From a code-sample perspective, this is in good shape.

The remaining issues are normal "we could improve this someday" items, not things that would make me think "ew, wrong way to do it."